

PRESENTATION

Ganesh Agrahari



WHAT EXACTLY WE NEED

◆ 1. Conversational Symptom Analysis

- The chatbot can engage in **multi-turn conversations** with patients to understand their symptoms in detail.
- It supports **mental health** queries (e.g., stress, anxiety, depression) as well as **physical symptoms** (e.g., fever, cold, body ache).
- Responds empathetically and provides **self-care suggestions** for mental and emotional well-being.

◆ 2. Prescription Recommendation System (*AI-augmented*)

- Based on the user's symptoms, the bot suggests **common medications** (e.g., paracetamol, cough syrups, pain relievers)
- along with:
 - Dosage guidance**
 - Usage frequency**
 - Precautionary advice**
- Designed for general conditions like **fever, cough, cold, pain, headache**, etc.
- Fine-tuned on Indian medical datasets for regional relevance.
- 🛡️ Note: Includes disclaimers to clarify that suggestions are not substitutes for professional medical advice.

◆ 3. Agentic AI Capabilities

- Can **schedule appointments** with doctors by integrating with a calendar, hospital management system, or third-party APIs.
- Smart prompts like:
"Book me a dermatologist for Saturday at 5 PM" → Triggers appointment booking flow.

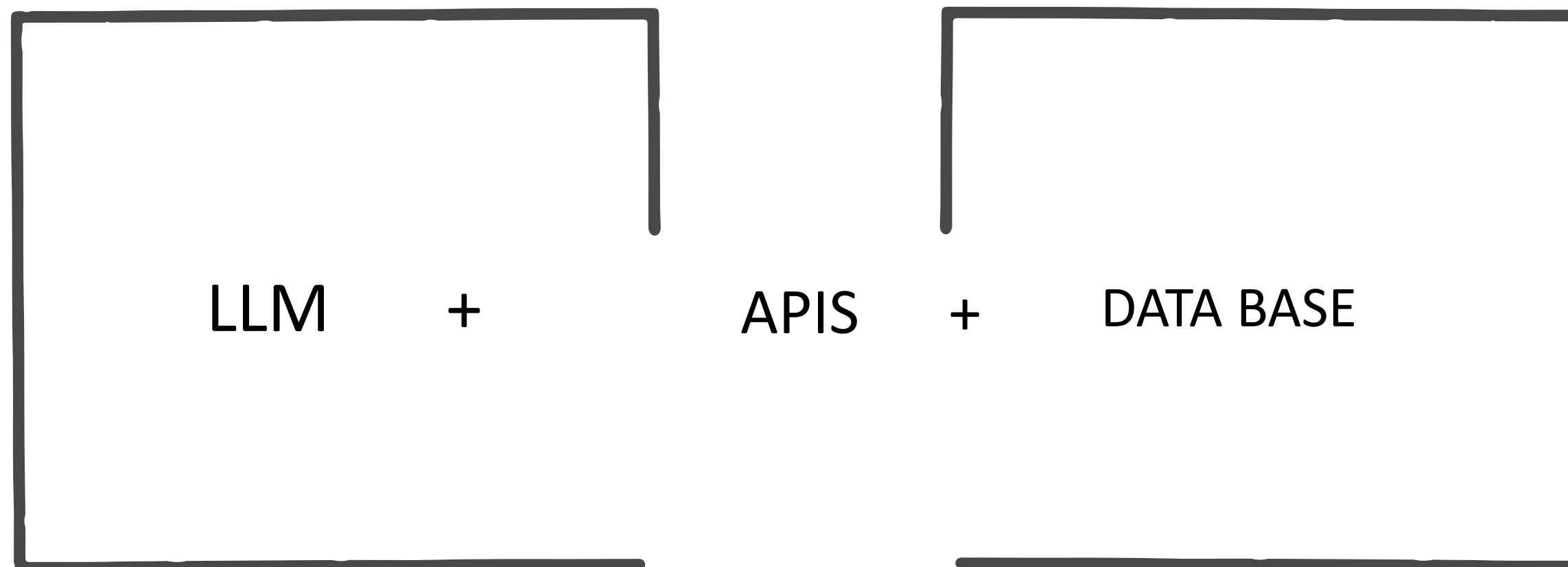
◆ 4. Location-Based Doctor Recommendations

- Automatically detects the user's **location (via GPS or manual input)**.
- Suggests **nearby verified doctors** based on the condition, time availability, and specialty.
- Integrates with platforms like **Practo, 1mg**, or custom databases for real-time doctor data.

◆ 5. Multilingual and Code-Mixed Input Handling

- Supports **English + Hindi + Hinglish** inputs to cater to the Indian audience.
- Example: *"Sir mujhe thoda bukhari ho raha hai aur gala dard hai"* → Still gives accurate prescription + advice.

The Apporch

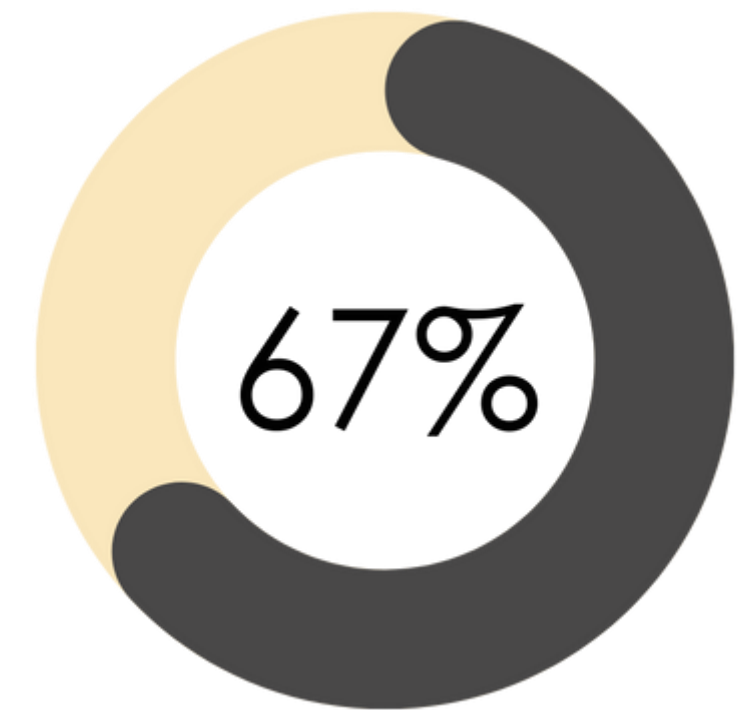


THE MAIN

- STEP1 : Choosing right model
- STEP2 : Tech Stacks
- STEP3 : Implementation

1.Choosing the Right Model

- For Conversational AI (Health Advice)**
 - Use **Med-PaLM 2** (Google Cloud) or **GPT-4-turbo** with fine-tuning on medical data.
 - Alternative: Train **BioGPT** or **Mistral-7B** on healthcare conversations
- For Prescription Suggestions**
 - Use **PubMedBERT** or **BioGPT**, fine-tuned on medical prescriptions.
 - Collect and preprocess a dataset containing symptoms, conditions, and prescriptions.
- For Agentic AI (Appointment Booking, Task Execution)**
 - Use **LangChain** or **AutoGen** to enable multi-step planning.
 - Integrate with **Google Calendar API** or **custom hospital APIs** for scheduling.
- For Location-Based Doctor Recommendations**
 - Use **Google Maps API** to find nearby clinics.
 - Store doctor profiles in **MongoDB** or **Firebase** with availability data.



2. Tech Stacks

Feature

Chatbot Core

Prescription Suggestion

Agentic AI (Appointments, Tasks)

Speech-to-Text (if needed)

Location-Based Doctor Search

Database for Users & Doctors

Frontend (Web/Mobile)

Tech/Model

OpenAI GPT-4 API, Med-PaLM 2, BioGPT

PubMedBERT, DrugBank API

LangChain, AutoGen

Whisper AI, Google Speech API

Google Maps API, Firebase

PostgreSQL / MongoDB

React.js / Flutter

Implementation

Step 1: Building the Conversational AI

- Fine-tune a **Med-PaLM-like model** on a dataset of medical FAQs and health conversations.
- Use **RAG (Retrieval-Augmented Generation)** to fetch trusted medical sources dynamically.

Step 2: Prescription Recommendation System

- Train a **small language model (like BioGPT)** on prescription guidelines.
- Use **structured rules** (like symptom-disease mappings).
- Add a **disclaimer** that it's not a replacement for a doctor.

Step 3: Agentic AI Capabilities

- Use **LangChain agents** to allow multi-step task execution.
- Example: If a patient asks, "Book me an appointment with a cardiologist," the AI:
 - Checks available doctors.
 - Confirms time slots.
 - Books via API integration.

Step 4: Location-Based Doctor Recommendations

- Integrate **Google Maps API** to fetch nearby hospitals/clinics.
- Store **verified doctor listings** in **MongoDB or Firebase**.

✅ Should You Build Your Own Small Language Model (SLM) for Prescription Recommendation?

🔍 Short Answer:

Start with a fine-tuned pretrained model (like BioGPT or PubMedBERT), **then build your own small model only if needed.**

💡 Why Start with Pretrained Models?

1. **Already trained on medical literature** (PubMed, clinical notes).
2. **Saves time and compute** – training your own model from scratch requires:
 1. High-quality medical data (symptoms ↔ disease ↔ medicine)
 2. GPUs or TPUs
 3. Time & expertise in deep learning
3. **Models like BioGPT or PubMedBERT** are specifically designed for biomedical NLP tasks.

🔨 When Should You Consider Building Your Own SLM?

Only if:

- You have **specific, structured datasets** (like local prescriptions from Indian doctors).
- You want to **compress the model** for faster inference (e.g., for mobile/web use).
- You need a **custom domain** (e.g., **Ayurveda, rural medicine**).

Then you can:

- Use **DistilBERT** or **TinyBioBERT**.
- Train it on a **CSV or JSON** of symptoms → diseases → medicine → dosage → precautions.

✓ Recommendation for Prescription Step

Approach

Tools

Pretrained + Fine-tuning

BioGPT, PubMedBERT + Hugging Face Transformers

Dataset

Open access sources + DrugBank + Medline + local prescription datasets

Model Hosting

Hugging Face Spaces / FastAPI backend

Frontend UI

React / Streamlit / Flutter



Can Pretrained Medical Models Work Well for Indian Context?

✓ Yes, but... with some caution.

✓ Why they help:

- Models like **BioGPT**, **PubMedBERT**, and **Med-PaLM** are trained on **global medical literature**, which includes standard diseases and treatments common in **India too**.
- They'll handle general issues like:
 - Fever, cough, cold, pain, infections
 - First-aid guidance
 - General medication suggestions (like paracetamol, ibuprofen)

⚠ Limitations for Indian Context

Here's where pretrained models may **struggle**:

Issue

Local brand medicine names

Ayurvedic / Homeopathy options

Language / Code-mixing (Hindi-English)

Cultural/Regional Health Advice

Why

These models won't know "Crocin," "Dolo 650," etc. unless you train them

Not present in foreign medical datasets

GPT/BERT models may not handle "Sir, thoda bukhar ho raha hai" very well

Like drinking kadha, turmeric milk, steam inhalation, etc.

💡 Your Best Strategy: Hybrid Approach

🔧 Step-by-Step:

1. **Start with a pretrained model** (BioGPT or PubMedBERT).
2. **Fine-tune it on a custom Indian prescription dataset:**
 1. Common medicines with Indian brand names
 2. Hindi-English mixed queries (synthetic data or real chat data)
 3. Local healthcare guidelines
3. Add a **symptom-to-prescription rule-based fallback** for critical cases (fever → paracetamol + dosage + advice).
4. Use **RAG (Retrieval-Augmented Generation)** to pull from trusted Indian medical sources like:
 1. AIIMS guidelines
 2. MedIndia, Practo, 1mg, etc.

✓ When Should You Build Your Own LLM or SLM?

👉 You should build your own SLM or LLM when:

Situation



Sensitive, private data (e.g., hospital records, EMRs)



Domain-specific language (e.g., Ayurveda, Indian prescriptions, Hinglish)



Extreme customization (e.g., different medical protocols)



Cost-effective at scale



Offline inference is needed



Edge/mobile deployment required



You're building proprietary IP

Why it makes sense

You can't risk data leaving the org (compliance: HIPAA, DISHA etc.)

Public models aren't trained on this style/data

You want to embed org-specific logic/flow

Constantly calling GPT-4 API gets expensive;
your own model = one-time training cost + free inference

Hospitals in rural areas may not have stable internet

You need a super light model (like a distilled BERT) to run on phones

For a product you want to license or patent,
own model = full control

✗ When You Should NOT Build Your Own Model (Yet)

Situation



Just building a prototype



Small user base (initial stage)



Lack of data scientists or ML infra



Limited budget or compute

Why not

Use GPT-4 or MedPaLM to validate idea fast

Not enough data to fine-tune yet

Training is costly + complex

GPT-4 API or Hugging Face Spaces is cheaper and faster to get started

Realistic Strategy

Since you're building a **healthcare chatbot**:

Phase

Strategy



Prototype

Use GPT-4 / BioGPT / MedPaLM with retrieval (e.g. LangChain + Docs)



Pilot/Test (real users)

Fine-tune BioGPT or create small model like TinyBioBERT with Indian data



Production (scale)

Train lightweight SLM (3B–7B) with custom Indian + internal data



Enterprise Deployment

Host it on your own server / mobile / edge for privacy & cost savings



Tools to Use If You Build Your Own:

- LoRA / QLoRA – low-cost fine-tuning
- DistilBERT or TinyLLM – smaller models
- Hugging Face – training + hosting
- ONNX or GGUF – convert models for mobile/edge



Step-by-Step: Fine-Tuning a Pretrained Medical Model



Use Case: Fine-tune BioGPT or PubMedBERT

Goal: Train it to give **Indian prescription suggestions** based on symptoms.



Step 1: Choose a Base Model



Good options:

- BioGPT
- PubMedBERT

You can use transformers library from Hugging Face to load these easily.



Step 2: Prepare the Dataset

Your dataset should look like this:

symptoms	diagnosis	medicine	dosage	advice
fever, headache	viral fever	paracetam ol	500mg twice daily	stay hydrated

Step 3: Choose Fine-Tuning Method

Option A: Full fine-tuning

-  Best performance
-  Expensive & needs GPU

Option B: LoRA or QLoRA

-  Lightweight
-  Can run on smaller GPU (like Google Colab or Kaggle)
-  Easy to deploy
-  Best for small companies and startups like QTech



Step 4: Set Up Training Environment

You'll need:

- transformers by Hugging Face
- datasets for loading your data
- peft for LoRA

- accelerate for GPU optimization

Basic training code available here:

👉 <https://github.com/huggingface/peft> (LoRA)

👉 <https://github.com/huggingface/transformers> (classic fine-tune)

-  **Approximate Cost of Fine-Tuning**
- 

Best value for QTech: LoRA / QLoRA
- 

Free option: Use Google Colab with T4 or A100 (Pro+ plan for ₹1.3k/month)

Option	Description	Infra Needed	Time	Cost
 LoRA on BioGPT (small dataset, ~10K records)	Most efficient	Google Colab Pro / Lambda Labs (1 x A100 or T4)	~2–3 hrs	~\$1–\$5
 Full Fine-tune (50K+ records)	High accuracy	A100 x1 or x2	~6–12 hrs	~\$20–\$40
 QLoRA on Hugging Face	With quantization (4-bit)	Free on HF with TGI	~2–4 hrs	Free (with HF grant)
 Hosted API (GPT-4, Claude, Gemini)	No training, just inference	None	Instant	\$0.003–\$0.03 per message



Thank you !